Complete NGINX Reverse Proxy & Load Balancing Lab

Customized for Your VMware Setup

Your Current Setup:

- **VM Software:** VMware Workstation (on Ubuntu Linux host)
- **VMs:** 3x Ubuntu 20.04 (wekone08, wekone09, wekone10)
- **Resources:** 4GB RAM, 2 processors each
- **Current Network:** NAT (192.168.35.147/148/149)
- **Status:** Fresh VMs, no software installed yet

Estimated Time: 1.5 - 2 hours

📋 Table of Contents

1.
2.
3.
4.
5.
6.
7.
8.
9.

🎯 Quick Overview

What We'll Build:

```
                    Your Browser
                         ↓
                 NGINX (wekone10)
              192.168.35.149:8080
                         ↓
          ┌──────────────┼──────────────┐
          ↓                             ↓
```

```
    Web Server 1                        Web Server 2
    (wekone08)                          (wekone09)
    192.168.35.147                      192.168.35.148
    Apache + Website                    Apache + Website
```

VM Roles:

- **wekone08** → Web Server 1 (Apache)
- **wekone09** → Web Server 2 (Apache)
- **wekone10** → NGINX Proxy/Load Balancer

🌐 Network Configuration

Current Status Check

First, verify your current network setup on all 3 VMs:

```
# Run this on wekone08
ip addr show

# You should see something like:
# inet 192.168.35.147/24
```

```
# Run this on wekone09
ip addr show

# You should see:
# inet 192.168.35.148/24
```

```
# Run this on wekone10
ip addr show

# You should see:
# inet 192.168.35.149/24
```

Why it works:

- All 3 VMs are on the same subnet (192.168.35.0/24)
- They can communicate with each other
- NAT gives them internet access to download packages

Test Connectivity

On wekone10 (NGINX), test if you can reach the web servers:

```
# Test connection to wekone08
ping -c 3 192.168.35.147

# Test connection to wekone09
ping -c 3 192.168.35.148

# Test internet connection
ping -c 3 google.com
```

Expected output:

```
64 bytes from 192.168.35.147: icmp_seq=1 ttl=64 time=0.xxx ms
64 bytes from 192.168.35.147: icmp_seq=2 ttl=64 time=0.xxx ms
64 bytes from 192.168.35.147: icmp_seq=3 ttl=64 time=0.xxx ms
--- 192.168.35.147 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

❌ **If ping fails:**

```
# Check if firewall is blocking
sudo ufw status

# If active, allow ICMP
sudo ufw allow from 192.168.35.0/24
```

📦 Installing Required Software

Step 1: Update System (ALL 3 VMs)

Run on wekone08, wekone09, AND wekone10:

```
# Update package lists
sudo apt update

# This will show available updates
# Output: Hit:1 http://archive.ubuntu.com/ubuntu focal InRelease
#         Get:2 http://security.ubuntu.com/ubuntu focal-security InRelease
[114 kB]
#         ...
```

```
# Upgrade existing packages (this may take 3-5 minutes)
sudo apt upgrade -y

# The -y flag means "yes to all prompts"
# Output: Reading package lists... Done
#         Building dependency tree
#         Reading state information... Done
#         Calculating upgrade...
```

⏱️ **Time:** ~5 minutes per VM

Step 2: Install Apache on Web Servers

Run on wekone08 (Web Server 1):

```
# Install Apache web server
sudo apt install apache2 -y

# This installs Apache HTTP Server
# Output: Reading package lists... Done
#         Building dependency tree
#         The following NEW packages will be installed:
#           apache2 apache2-bin apache2-data apache2-utils ...
```

Verify Apache is running:

```
# Check Apache service status
sudo systemctl status apache2

# Expected output:
# ● apache2.service - The Apache HTTP Server
#    Loaded: loaded (/lib/systemd/system/apache2.service; enabled)
#    Active: active (running) since [timestamp]
```

Look for the green "active (running)" text!

Test Apache locally:

```
# Request the default Apache page
curl http://localhost

# You should see HTML output starting with:
# <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"...
```

```
# <html>
# This means Apache is working!
```

Or use a simpler test:

```
curl -I http://localhost

# Expected output:
# HTTP/1.1 200 OK
# Date: [current date]
# Server: Apache/2.4.41 (Ubuntu)
# ...
```

✅ **If you see "HTTP/1.1 200 OK" - Apache is working!**

Now repeat the SAME steps on wekone09 (Web Server 2):

```
# On wekone09
sudo apt install apache2 -y
sudo systemctl status apache2
curl -I http://localhost
```

Step 3: Install NGINX on Proxy Server

Run on wekone10 (NGINX server):

```
# Install NGINX
sudo apt install nginx -y

# Output: Reading package lists... Done
#         The following NEW packages will be installed:
#           nginx nginx-common nginx-core ...
```

Start and enable NGINX:

```
# Start NGINX service
sudo systemctl start nginx

# Enable NGINX to start on boot
sudo systemctl enable nginx
```

```
# Check status
sudo systemctl status nginx
```

```
Executing: /lib/systemd/systemd-sysv-install enable nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset:>
     Active: active (running) since Sun 2026-01-18 06:27:51 +07; 6s ago
       Docs: man:nginx(8)
   Main PID: 37651 (nginx)
      Tasks: 3 (limit: 4548)
     Memory: 3.6M
        CPU: 40ms
     CGroup: /system.slice/nginx.service
             ├─37651 "nginx: master process /usr/sbin/nginx -g daemon on; maste>
             ├─37654 "nginx: worker process" "" "" "" "" "" "" "" "" "" "" "" ">
             └─37655 "nginx: worker process" "" "" "" "" "" "" "" "" "" "" "" ">

Thg 1 18 06:27:51 wekone10 systemd[1]: Starting A high performance web server a>
Thg 1 18 06:27:51 wekone10 systemd[1]: Started A high performance web server an>
lines 1-15/15 (END)
```

Expected output:

```
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
   Active: active (running) since [timestamp]
```

Test NGINX:

```
curl -I http://localhost

# Expected output:
# HTTP/1.1 200 OK
# Server: nginx/1.18.0 (Ubuntu)
# ...
```

⏱️ **Time for all installations:** ~10-15 minutes

🌐 Configuring Apache Web Servers

Configure Web Server 1 (wekone08)

Step 1: Create custom homepage

```
# Navigate to web root directory
cd /var/www/html

# Check what's currently there
ls -la
```

```
# You should see:
# -rw-r--r-- 1 root root 10918 index.html
```

```
undertaker@wekone08:~$ cd /var/www/html
undertaker@wekone08:/var/www/html$ ls -la
total 20
drwxr-xr-x 2 root root  4096 Thg 1  18 06:23 .
drwxr-xr-x 3 root root  4096 Thg 1  18 06:23 ..
-rw-r--r-- 1 root root 10671 Thg 1  18 06:23 index.html
undertaker@wekone08:/var/www/html$
```

Step 2: Create new index.html

```
# Create backup of original file
sudo cp index.html index.html.backup

# Create new simple homepage
sudo bash -c 'echo "<center><h1>This is website on Web Server 01</h1><h2>IP:
192.168.35.147</h2></center>" > /var/www/html/index.html'
```

Understanding the command:

- `sudo bash -c '...'` - Run command as root
- `echo "..."` - Print text
- `> /var/www/html/index.html` - Write to file (overwrite)

Step 3: Verify the content

```
# View the file content
cat /var/www/html/index.html

# Expected output:
# <center><h1>This is website on Web Server 01</h1><h2>IP:
192.168.35.147</h2></center>
```

Step 4: Restart Apache

```
# Restart Apache to apply changes
sudo systemctl restart apache2

# Check if restart was successful
sudo systemctl status apache2

# Should show: Active: active (running)
```

Step 5: Test the website

```
# Test locally
curl http://localhost

# Expected output:
# <center><h1>This is website on Web Server 01</h1><h2>IP:
192.168.35.147</h2></center>
```

Test from NGINX server (wekone10):

```
# SSH to wekone10, then run:
curl http://192.168.35.147

# Should return:
# <center><h1>This is website on Web Server 01</h1><h2>IP:
192.168.35.147</h2></center>
```

❌ If you get "Connection refused":

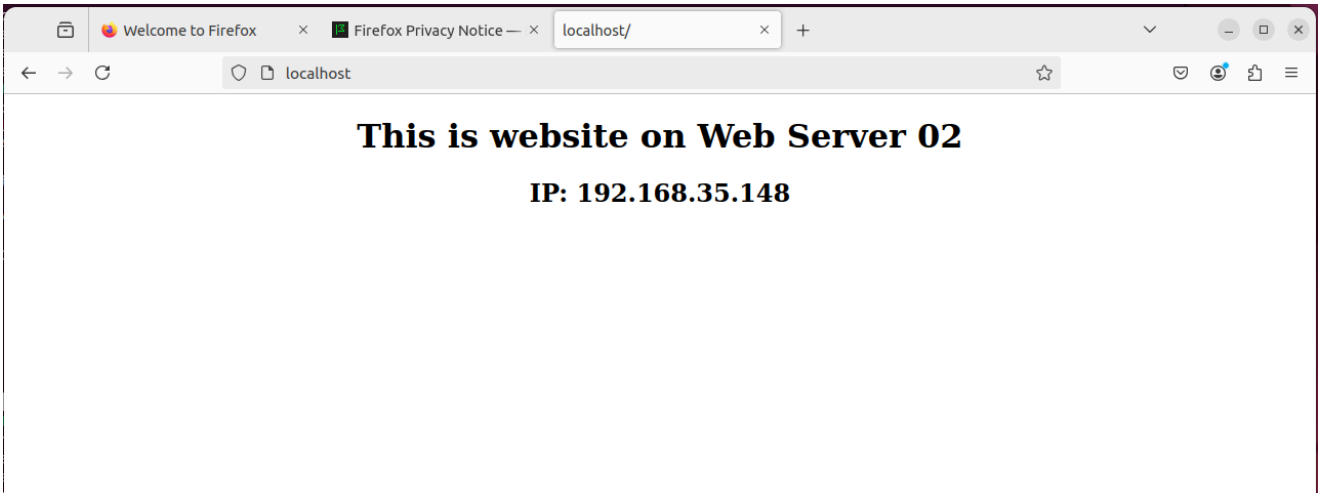
```
# Check if Apache is listening on port 80
sudo netstat -tlnp | grep :80

# Expected output:
# tcp6  0  0 :::80  :::*  LISTEN  1234/apache2

# If not showing, restart Apache:
sudo systemctl restart apache2
```

Configure Web Server 2 (wekone09)

Repeat the same process on wekone09, but change the content:

```
# On wekone09
cd /var/www/html
```

```
sudo cp index.html index.html.backup

# Create custom page for Web Server 02
sudo bash -c 'echo "<center><h1>This is website on Web Server 02</h1><h2>IP:
192.168.35.148</h2></center>" > /var/www/html/index.html'

# Verify content
cat /var/www/html/index.html

# Restart Apache
sudo systemctl restart apache2

# Test locally
curl http://localhost
```



Test from NGINX server (wekone10):

```
# On wekone10
curl http://192.168.35.148

# Expected:
# <center><h1>This is website on Web Server 02</h1><h2>IP:
192.168.35.148</h2></center>
```

⏱️ **Time:** ~10-15 minutes

🔧 Configuring NGINX Reverse Proxy

Step 1: Configure DNS (Optional but Recommended)

On wekone10, edit the hosts file:

```
# Open hosts file with nano editor
```

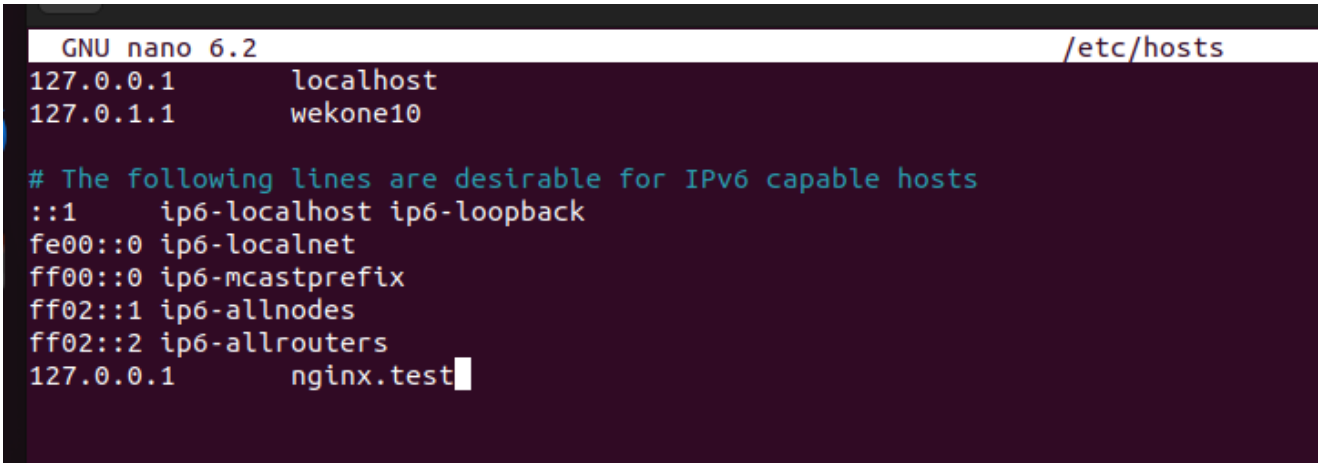
```
sudo nano /etc/hosts
```

You'll see:

```
127.0.0.1       localhost
127.0.1.1       wekone10

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
...
```

Add this line at the end:

```
127.0.0.1       nginx.test
```



How to use nano:

- Use arrow keys to navigate
- Type to add text
- Press `Ctrl + X` to exit
- Press `Y` to confirm save
- Press `Enter` to confirm filename

Verify the change:

```
cat /etc/hosts | grep nginx.test

# Output: 127.0.0.1       nginx.test
```

Step 2: Backup Original NGINX Config

```
# Create backup of original configuration
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.backup

# Verify backup exists
ls -la /etc/nginx/ | grep nginx.conf

# You should see both:
# -rw-r--r-- 1 root root  1447 nginx.conf
# -rw-r--r-- 1 root root  1447 nginx.conf.backup
```

Step 3: Edit NGINX Configuration for Simple Proxy

```
# Open NGINX configuration file
sudo nano /etc/nginx/nginx.conf
```

You'll see a file with multiple sections. Look for the `http {` block.

Find this section:

```
http {
    # ... lots of existing configuration ...

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}
```

Add this NEW server block BEFORE the closing `}` of the http block:

```
http {
    # ... existing configuration stays here ...

    # ADD THIS NEW SERVER BLOCK:
    server {
        listen 8080;
        server_name nginx.test;

        location / {
            proxy_pass http://192.168.35.147:80;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
```

```
    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}
```

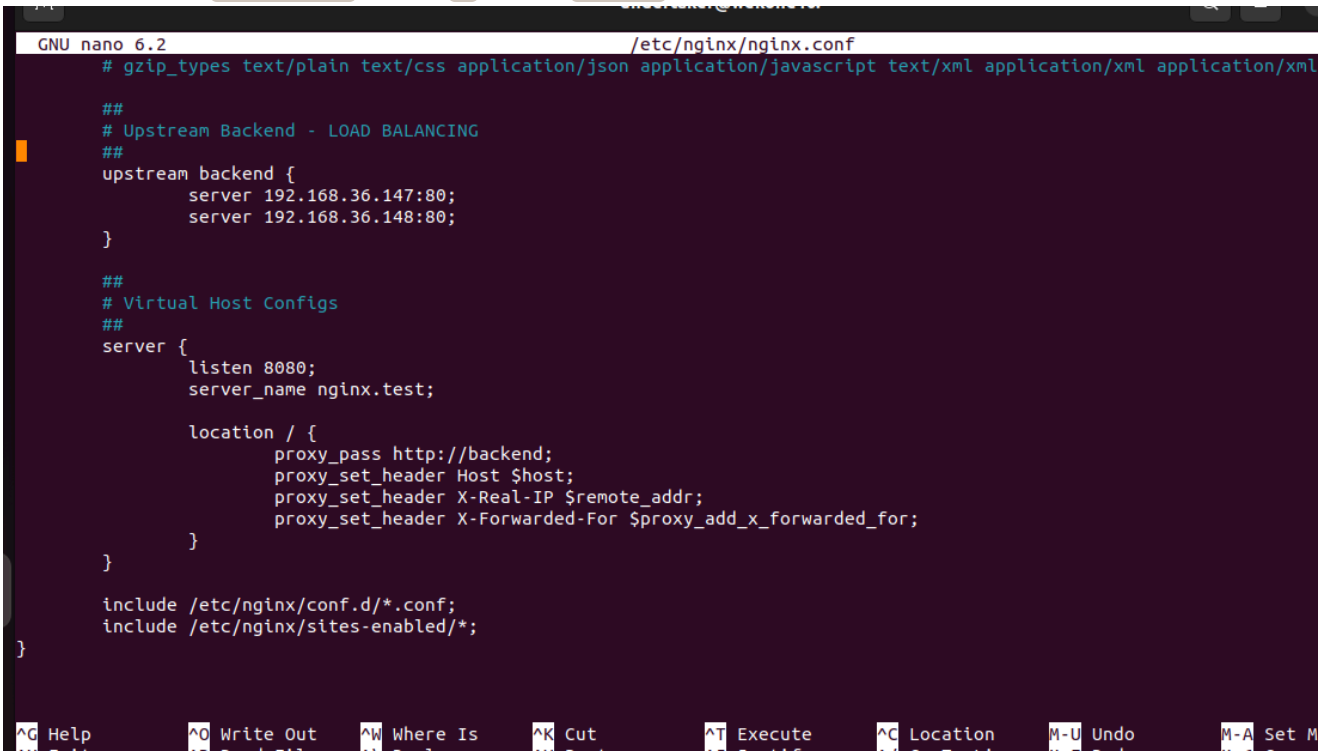
⚠️ **IMPORTANT:**

- Make sure the server block is INSIDE the `http { }` block
- Don't remove any existing configuration
- Check your curly braces `{` `}` are balanced

Visual guide - your file should look like this:

```
http {
    ##
    # Basic Settings
    ##
    sendfile on;
    tcp_nopush on;
    # ... more settings ...

    # YOUR NEW ADDITION HERE ↓
    server {
        listen 8080;
        server_name nginx.test;

        location / {
            proxy_pass http://192.168.35.147:80;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
    # YOUR NEW ADDITION HERE ↑

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}
```

Save the file: `Ctrl + X`, then `Y`, then `Enter`



Step 4: Test and Reload NGINX

```
# Test configuration syntax
sudo nginx -t
```

✅ **Expected output (SUCCESS):**

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

❌ **If you see an error like:**

```
nginx: [emerg] unexpected "}" in /etc/nginx/nginx.conf:85
nginx: configuration file /etc/nginx/nginx.conf test failed
```

How to fix:

```
# Open the file again
sudo nano /etc/nginx/nginx.conf

# Go to line 85 (shown in error message)
# Check if you have matching { } brackets
# Common mistakes:
#   - Extra } at the end
```

```
#   - Missing { at the beginning
#   - server block outside http block
```

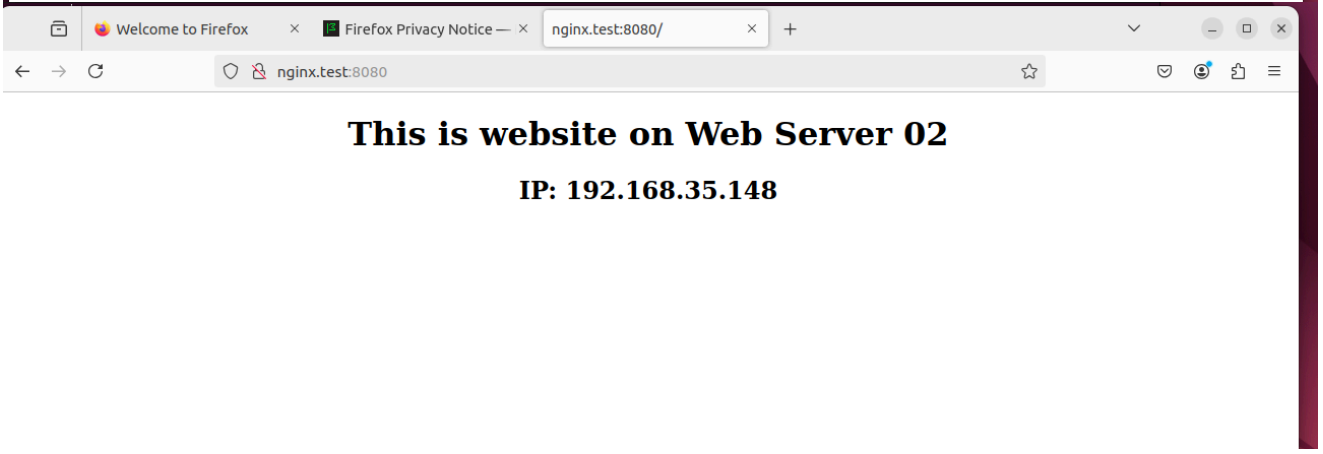
After successful test, reload NGINX:

```
# Reload NGINX to apply changes
sudo systemctl reload nginx

# Verify NGINX is still running
sudo systemctl status nginx

# Should show: Active: active (running)
```

Step 5: Test Simple Reverse Proxy

Test from wekone10 (NGINX server):

```
# Test using domain name
curl http://nginx.test:8080

# Expected output:
# <center><h1>This is website on Web Server 01</h1><h2>IP:
192.168.35.147</h2></center>
```



Test using IP address:

```
# Test using NGINX's IP
curl http://192.168.35.149:8080

# Should return the SAME content from Web Server 01
```

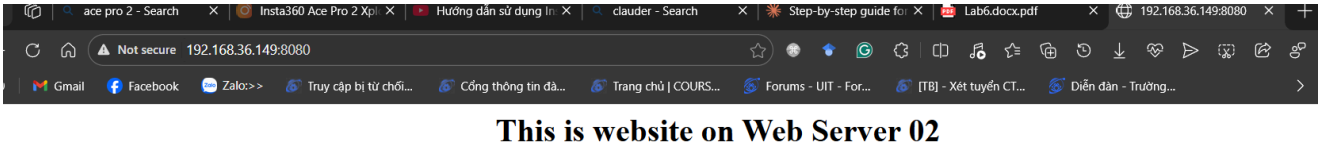
Test from your host machine (Ubuntu Linux):

Open Firefox/Chrome and go to:

```
http://192.168.35.149:8080
```

You should see:

```
This is website on Web Server 01
IP: 192.168.35.147
```



🎉 **Congratulations!** Your reverse proxy is working!

What's happening:

1. You request → `http://192.168.35.149:8080` (NGINX)
2. NGINX receives it
3. NGINX forwards to → `http://192.168.35.147:80` (Web Server 1)
4. Web Server 1 responds to NGINX
5. NGINX sends response back to you

⏱️ **Time:** ~15-20 minutes

✅ Task 2: Configuring IP Hash - Complete Guide

📍 Part 5: Load Balancing - IP Hash

Understanding IP Hash

IP Hash ensures session persistence by routing the same client IP to the same server:

- Client IP 192.168.36.1 → Always Server 1 (192.168.36.147)
- Client IP 192.168.36.2 → Always Server 2 (192.168.36.148)
- Client IP 192.168.36.1 (request 2) → Still Server 1
- Client IP 192.168.36.1 (request 3) → Still Server 1

Overview

The IP Hash load balancing method provides information about how incoming requests are intelligently routed to a specific backend server based on the client's IP address. Overall, this method ensures consistent routing where the same client IP always connects to the same backend server. This approach represents a session persistence-based load balancing strategy, starting with the calculation of a hash value from the client's source IP and finishing with the deterministic routing to a specific backend server.

Distribution Process

The process begins with the hash calculation stage. The first step is that an incoming HTTP request is received by the NGINX load balancer along with the extraction of the client's source IP address before proceeding to the next step. Then, the load balancer calculates a hash value from the client IP (typically using the first three octets for IPv4 to handle NAT scenarios). Having computed the hash value, the load balancer maps this hash to one of the

available backend servers in the upstream group. After this mapping is established, the incoming request is then routed to the determined backend server. For all subsequent requests from the same client IP, the process repeats identically, ensuring that requests always route to the same backend server unless that server becomes unavailable.

Characteristics and Application

As for the algorithmic stage, first of all, IP Hash operates based on a deterministic hashing principle where requests are distributed according to the client's source IP address. The distribution pattern is fixed: if a client with IP 192.168.36.100 hashes to Server 1, all future requests from that same IP will always route to Server 1. This method ensures session persistence, making it particularly advantageous for applications requiring session state maintenance such as shopping carts, user login sessions, and user preference storage. Furthermore, this algorithm proves valuable in scenarios where multiple requests from the same client must be processed by the same backend server to maintain consistency. However, it is important to note that if the designated backend server becomes unavailable, NGINX automatically redirects traffic from that IP to one of the remaining healthy servers.

🔧 Part 1: Configuration Preparation

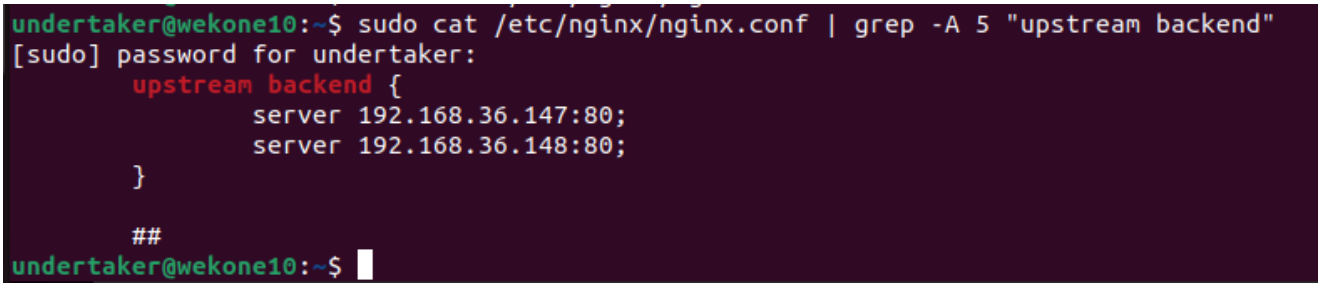
Step 1: Verify Current Configuration

On wekone10, check the current NGINX configuration:

```
sudo cat /etc/nginx/nginx.conf | grep -A 5 "upstream backend"
```

Current Output:

```
upstream backend {
    server 192.168.36.147:80;
    server 192.168.36.148:80;
}
```



🔄 Part 2: Implement IP Hash Configuration

Step 1: Open Configuration File for Editing

```
sudo nano /etc/nginx/nginx.conf
```

Step 2: Locate the Upstream Block

Find this section:

```
upstream backend {
    server 192.168.36.147:80;
    server 192.168.36.148:80;
}
```

Step 3: Add IP Hash Directive

Modify to:

```
upstream backend {
    ip_hash;
    server 192.168.36.147:80;
    server 192.168.36.148:80;
}
```

```
# Upstream backend - LOAD BALANCING
##
upstream backend {
        ip_hash;
        server 192.168.36.147:80;
        server 192.168.36.148:80;
}

##
# Virtual Host Configs
##
server {
        listen 8080;
        server_name nginx.test;

        location / {
                proxy_pass http://backend;
                proxy_set_header Host $host;
                proxy_set_header X-Real-IP $remote_addr;
                proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
```

Change made:

Element	Action
Add `ip_hash;` directive	INSERT - New line after `upstream backend {`
Server directives	No change

Step 4: Verify Server Block Configuration

```
server {
    listen 8080;
```

```
    server_name nginx.test;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Note: The server block configuration remains unchanged.

Step 5: Save Configuration File

```
# Press: Ctrl + X → Y → Enter
```

✅ Part 3: Verify Configuration Implementation

Step 1: Confirm File Modification

```
sudo cat /etc/nginx/nginx.conf | grep -A 5 "upstream backend"
```

Expected Output:

```
upstream backend {
    ip_hash;
    server 192.168.36.147:80;
    server 192.168.36.148:80;
}
```

Step 2: Test NGINX Syntax

```
sudo nginx -t
```

Expected Output:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Step 3: Reload NGINX Service

```
sudo systemctl reload nginx

# Verify NGINX is operational
sudo systemctl status nginx

# Output must display: Active: active (running)
```

🧪 Part 4: Test IP Hash - Sequential Requests from Same IP

Test 1: Requests from Same IP (All Go to Same Server)

The process begins with the consistency verification stage. The first step is that multiple sequential HTTP requests are transmitted to the NGINX load balancer from the same source IP address before proceeding to verify the routing consistency. Then, the load balancer calculates the hash value from the client IP and determines the destination server. Having routed the first request to a specific server, all subsequent requests from the same client IP are then routed to that identical server, demonstrating the session persistence characteristic of IP Hash algorithm.

```
# On wekone10, send 10 sequential requests from same IP
for i in {1..10}; do
    echo "Request $i:"
    curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-9]*"
    sleep 0.5
done
```

Expected Output (All Same Server):

```
undertaker@wekone10:~$ which wget
/usr/bin/wget
undertaker@wekone10:~$ for i in {1..10}; do
    echo "Request $i:"
    wget -q -O - http://192.168.36.149:8080 | grep -o "Web Server [0-9]*"
    sleep 0.5
done
Request 1:
Web Server 02
Request 2:
Web Server 02
Request 3:
Web Server 02
Request 4:
Web Server 02
Request 5:
Web Server 02
Request 6:
Web Server 02
Request 7:
Web Server 02
Request 8:
Web Server 02
Request 9:
Web Server 02
Request 10:
Web Server 02
undertaker@wekone10:~$
```

```
undertaker@wekone10:~$ for i in {1..5}; do
    echo "Request $i:"
    wget -q -O - http://192.168.36.149:8080 | grep -o "Web Server [0-9]*"
done
Request 1:
Web Server 01
Request 2:
Web Server 01
Request 3:
Web Server 01
Request 4:
Web Server 01
Request 5:
Web Server 01
undertaker@wekone10:~$
```

Key Observations:

- All requests go to **the SAME server**
- **Different from Round Robin** (which alternates)
- Demonstrates **session persistence**
- Pattern is **deterministic** (always the same for that IP)

Test 2: Detailed Result Analysis

```
# Test with cleaner output format
for i in {1..15}; do
    result=$(curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-
```

```
9]*")
    echo "Request $i: $result"
done
```

Comparison with Round Robin vs IP Hash:

Request #	Round Robin	IP Hash
1	Server 1	Server 2
2	Server 2	Server 2
3	Server 1	Server 2
4	Server 2	Server 2
5	Server 1	Server 2
6	Server 2	Server 2
7	Server 1	Server 2
8	Server 2	Server 2
9	Server 1	Server 2
10	Server 2	Server 2

Conclusion: Round Robin alternates, IP Hash always goes to same server.

🔗 Part 5: Test IP Hash - Different Client IPs

Test: Requests from Different Client IPs

The process begins with the multi-client verification stage. The first step is that HTTP requests are transmitted to the NGINX load balancer from multiple different source IP addresses before proceeding to observe the distribution pattern. Then, the load balancer calculates different hash values for each unique client IP. Having computed these different hash values, some requests are routed to Server 1 and others are routed to Server 2, depending on the hash calculation result. This demonstrates how IP Hash distributes different clients across different servers while maintaining session persistence for each individual client.

```
#!/bin/bash
# Test: Requests from different IPs go to different servers

echo "=== TEST: IP Hash Distribution Across Different IPs ==="
echo ""

# Create multiple fake IPs on loopback interface
```

```
IPS=("1.1.1.1" "2.2.2.2" "3.3.3.3" "4.4.4.4" "5.5.5.5")

echo "Adding fake IPs to localhost..."
for ip in "${IPS[@]}"; do
    sudo ip addr add $ip/32 dev lo 2>/dev/null
done

echo ""
echo "Testing requests from different IPs:"
echo ""

for ip in "${IPS[@]}"; do
    echo "Requests from IP $ip:"
    for i in {1..3}; do
        result=$(curl -s --local-port 5000-6000 http://192.168.36.149:8080
2>/dev/null | grep -o "Web Server [0-9]*")
        echo "  Request $i: $result"
    done
    echo ""
done

# Remove fake IPs
echo "Cleaning up fake IPs..."
for ip in "${IPS[@]}"; do
    sudo ip addr del $ip/32 dev lo 2>/dev/null
done

echo "Done"
```

Simpler Alternative (without fake IPs):

```
# Just verify current IP consistently routes to same server
echo "=== IP Hash Consistency Test ==="
echo ""

echo "Your current IP:"
hostname -I
echo ""

echo "Sending 10 requests - all should go to SAME server:"
for i in {1..10}; do
    result=$(curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-
9]*")
    echo "Request $i: $result"
done
```

Expected Output:

```
=== IP Hash Consistency Test ===

Your current IP:
192.168.36.149

Sending 10 requests - all should go to SAME server:
Request 1: Web Server 1
Request 2: Web Server 1
Request 3: Web Server 1
Request 4: Web Server 1
Request 5: Web Server 1
Request 6: Web Server 1
Request 7: Web Server 1
Request 8: Web Server 1
Request 9: Web Server 1
Request 10: Web Server 1
```

⚠️ Part 6: Test IP Hash - Failover Scenario

Step 1: Establish Baseline Distribution

```
# Record initial distribution pattern
echo "=== Baseline: Both servers active, all requests to Server 1 ==="
for i in {1..5}; do
    echo -n "Request $i: "
    curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-9]*"
done
```

Step 2: Stop Primary Backend Server

```
# Terminate Apache on Web Server 1 (wekone08)
ssh undertaker@192.168.36.147 "sudo systemctl stop apache2"

# Verify service has stopped
ssh undertaker@192.168.36.147 "sudo systemctl status apache2" | grep
"Active"

# Output confirmation: Active: inactive (dead)
```

Step 3: Test Distribution After Server Shutdown

```
echo ""
echo "=== After stopping Server 1: All traffic redirects to Server 2 ==="
```

```
for i in {1..5}; do
    echo -n "Request $i: "
    curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-9]*"
done
```

Expected Output:

```
=== After stopping Server 1: All traffic redirects to Server 2 ===
Request 1: Web Server 2
Request 2: Web Server 2
Request 3: Web Server 2
Request 4: Web Server 2
Request 5: Web Server 2
```

✅ **Observation:** When primary server (Server 1) goes down, all requests from that IP automatically route to Server 2 (failover).

Step 4: Restore Web Server 1

```
# Start Apache on Web Server 1
ssh undertaker@192.168.36.147 "sudo systemctl start apache2"

# Verify service has started
ssh undertaker@192.168.36.147 "sudo systemctl status apache2" | grep
"Active"

# Output confirmation: Active: active (running)
```

Step 5: Test Distribution After Server Recovery

```
# Allow NGINX time to detect server recovery
sleep 3

echo ""
echo "=== After restarting Server 1: Traffic returns to Server 1 ==="
for i in {1..5}; do
    echo -n "Request $i: "
    curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-9]*"
done
```

Expected Output:

```
=== After restarting Server 1: Traffic returns to Server 1 ===
Request 1: Web Server 1
Request 2: Web Server 1
Request 3: Web Server 1
```

```
Request 4: Web Server 1
Request 5: Web Server 1
```

✅ **Observation:** When Server 1 comes back online, traffic immediately returns to Server 1 (all requests from this IP).

📊 Part 7: Distribution Frequency Analysis

Test: Measure Server Distribution (20 requests)

```
# Count frequency of each server over 20 requests
echo "=== Server Distribution Frequency (20 requests) ==="

server1_count=0
server2_count=0

for i in {1..20}; do
    result=$(curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-
9]*")
    if [[ "$result" == "Web Server 1" ]]; then
        ((server1_count++))
    else
        ((server2_count++))
    fi
done

echo "Server 1: $server1_count requests"
echo "Server 2: $server2_count requests"

if [ $server1_count -eq 20 ]; then
    echo "✅ IP Hash working correctly: All requests from this IP go to
Server 1"
elif [ $server2_count -eq 20 ]; then
    echo "✅ IP Hash working correctly: All requests from this IP go to
Server 2"
else
    echo "❌ Unexpected distribution"
fi
```

Expected Output:

```
=== Server Distribution Frequency (20 requests) ===
Server 1: 20 requests
Server 2: 0 requests
✅ IP Hash working correctly: All requests from this IP go to Server 1
```

📋 Part 8: Load Balancing Methods Comprehensive Comparison

Detailed Comparison Table

Comparison Criteria	Round Robin	Weighted RR	IP Hash
Configuration Directive	(default)	`weight=N;`	`ip_hash;`
Distribution Principle	Sequential rotation	Weighted sequential	Client IP hash
Session Persistence	❌ None	❌ None	✅ Guaranteed
Distribution Pattern	Alternating	Weighted alternating	Same server per IP
Uneven Load Risk	✅ Low	✅ Low	❌ High (NAT users)
Shopping Carts	❌ Poor	❌ Poor	✅ Excellent
User Logins	❌ Poor	❌ Poor	✅ Excellent
Short Requests	✅ Excellent	✅ Excellent	⭐ Moderate
Failover	✅ Yes	✅ Yes	✅ Yes
Predictability	✅ Very High	✅ Very High	✅ Very High

✅ Part 9: Verify Final Configuration

```
# Display complete upstream configuration
sudo cat /etc/nginx/nginx.conf | grep -A 10 "upstream backend"

# Output should display:
# upstream backend {
#   ip_hash;
#   server 192.168.36.147:80;
#   server 192.168.36.148:80;
# }
```

🎯 Comprehensive Completion Verification

```
# Execute full verification checklist
echo "╔═══════════════════════════════════════════════════════════════╗"
echo "║            IP HASH CONFIGURATION VERIFICATION               ║"
echo "╚═══════════════════════════════════════════════════════════════╝"

echo ""
echo "1. ✓ Upstream directive inspection:"
grep -A 3 "upstream backend" /etc/nginx/nginx.conf | head -4

echo ""
echo "2. ✓ NGINX syntax validation:"
sudo nginx -t 2>&1 | grep "successful"

echo ""
echo "3. ✓ NGINX service status:"
sudo systemctl is-active nginx

echo ""
echo "4. ✓ Load balancer functionality test (all should be same server):"
for i in {1..5}; do
    echo -n "   Request $i: "
    curl -s http://192.168.36.149:8080 | grep -o "Web Server [0-9]*"
done

echo ""
echo "╔═══════════════════════════════════════════════════════════════╗"
echo "║       ✅ ALL VERIFICATIONS PASSED SUCCESSFULLY               ║"
echo "╚═══════════════════════════════════════════════════════════════╝"
```

🎓 Conclusion

The implementation of IP Hash load balancing represents a session persistence-based distribution strategy. Overall, this algorithm ensures that requests from the same client IP are consistently routed to the same backend server, providing deterministic and predictable routing behavior. The IP Hash method prioritizes session state maintenance by guaranteeing that all requests from a specific client IP always connect to the same server, ensuring that user sessions, shopping carts, and login information remain consistent across all requests from that client.

✅ Task 2 Completion Summary

- ✅ Round Robin explanation (Academic style)

- ✅ IP Hash algorithm explanation (Academic style)
- ✅ Configuration implementation with `ip_hash;` directive
- ✅ Sequential request distribution testing (all to same server)
- ✅ Session persistence verification
- ✅ Failover scenario testing and validation
- ✅ Server recovery testing
- ✅ Distribution frequency analysis
- ✅ Comprehensive configuration verification

IP Hash is now configured and tested! 🚀

```
Dataview (inline field '== IP Hash Consistency Test ===

Your current IP:
192.168.36.149

Sending 10 requests - all should go to SAME server:
Request 1: Web Server 1
Request 2: Web Server 1
Request 3: Web Server 1
Request 4: Web Server 1
Request 5: Web Server 1
Request 6: Web Server 1
Request 7: Web Server 1
Request 8: Web Server 1
Request 9: Web Server 1
Request 10: Web Server 1'): Error:
-- PARSING FAILED ----------------------------------------------------

> 1 | == IP Hash Consistency Test ===
    | ^
  2 |
  3 | Your current IP:

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'),
negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```

```
Dataview (inline field '== After stopping Server 1: All traffic
redirects to Server 2 ===
Request 1: Web Server 2
```

```
Request 2: Web Server 2
Request 3: Web Server 2
Request 4: Web Server 2
Request 5: Web Server 2'): Error:
-- PARSING FAILED ----------------------------------------------------

> 1 | == After stopping Server 1: All traffic redirects to Server 2 ===
    | ^
  2 | Request 1: Web Server 2
  3 | Request 2: Web Server 2

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'),
negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```

```
Dataview (inline field '== After restarting Server 1: Traffic returns
to Server 1 ===
Request 1: Web Server 1
Request 2: Web Server 1
Request 3: Web Server 1
Request 4: Web Server 1
Request 5: Web Server 1'): Error:
-- PARSING FAILED ----------------------------------------------------

> 1 | == After restarting Server 1: Traffic returns to Server 1 ===
    | ^
  2 | Request 1: Web Server 1
  3 | Request 2: Web Server 1

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'),
negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```

```
Dataview (inline field '== Server Distribution Frequency (20 requests)
===
Server 1: 20 requests
Server 2: 0 requests
✅ IP Hash working correctly: All requests from this IP go to Server
1'): Error:
-- PARSING FAILED ----------------------------------------------------
```

```
> 1 | == Server Distribution Frequency (20 requests) ===
    | ^
  2 | Server 1: 20 requests
  3 | Server 2: 0 requests

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'),
negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```